

Informe tecnico - Primera entrega parcial del TPI (Food Store)

Fabri - Base de Datos II - PostgreSQL 18 + DBeaver (Manual). Repo con TP1-TP5 + tpi-parcial.

1. Que se implemento por unidad

TP1 (food-store-tp1/): ER, relacional con N:M en pedido_producto, 3FN/BCNF, DDL con ENUM forma_pago_enum, IDENTITY, TIMESTAMPTZ, PK/FK justificadas, UNIQUE, CHECKs y flags de baja logica. TP2: reglas R1/R2 (funciones + triggers), 3 escenarios de concurrencia, protocolo copia-transaccion-respaldo. TP3-TP5: carga 200k, indexado con EXPLAIN, vistas + MV, ventanas y anti-joins. Cierre (tpi-parcial/): HAVING, PROCEDURE + CALL, demo de borrado.

2. Como se probó

Siempre en food_store_copia, Manual, respaldo previo. DDL con COMMIT; experimentos con ROLLBACK. Spec Kiro, generacion OpenCode, lectura linea por linea, EXCEPT en ambos sentidos.

3. Resultados por objetivo (checklist de 9)

Obj	Evidencia	Resultado
1. ER	diagrama TP1	5 entidades, N:M detectada
2. Relacional	Food Store.sql	intermedia + FKs justificadas
3. 3FN/BCNF	desarrollo TP1	DF documentadas
4. DDL	schema + indices	ENUM/IDENTITY/TZ/UNIQUE
5. DML	queries + h1	JOIN, GROUP BY, HAVING (6 filas), EXISTS, RANK
6. Vistas/func/proc	views, triggers, b0	3 vistas + MV + 2 func + 1 proc
7. Reglas	restricciones.sql	CHECK/UNIQUE/triggers R1-R2
8. Transacciones	informe_concurrencia	3 anomalias reproducidas
9. Borrado logico	b1/b2/b3	CALL: 50001/0 filas; ROLLBACK: 50002

4. Optimizaciones antes/despues

Consulta	Antes	Despues	X
TP3 Q3 precio	Seq 3.394 ms	Bitmap 1.636 ms	2.07x
TP3 Q1	Bitmap 2.101 ms	Bitmap nuevo 1.430 ms	1.46x
TP4 Q-A/Q-B	Hash 430/152.9 ms	sigue Hash	no efectivas
TP5 Q1	Seq 43.185 ms	Index 0.031 ms	1393x
TP5 Q2	Bitmap+Filter 9.581 ms	Index Cond 0.248 ms	38.6x
TP5 Q3	Seq 19.779 ms	sigue Seq 14.065 ms	no efectiva
MV facturacion	~430 ms	0.038 ms	~11300x

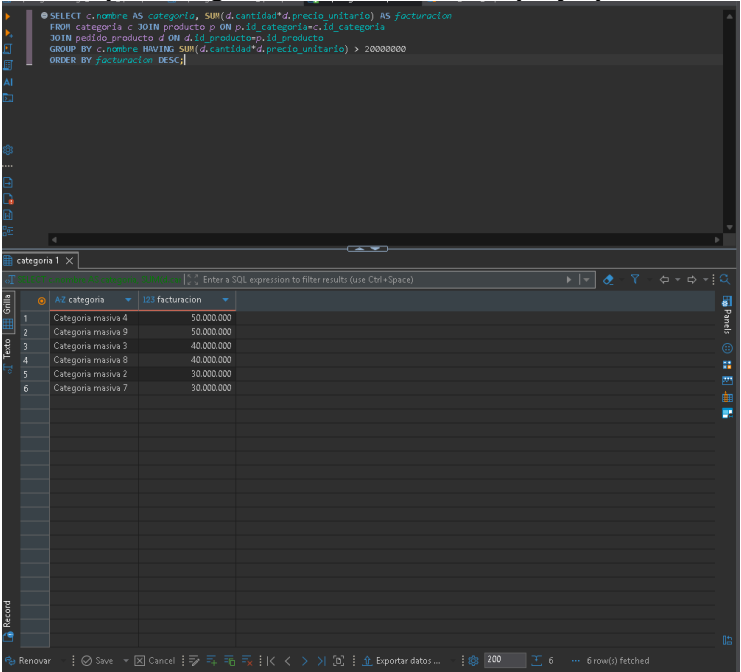
5. Uso de IA

Kiro (especificar) + OpenCode (generar), sin otras IA. Aceptados: indices compuestos/parciales, vistas, MV, procedimiento. Descartados: indice en forma_pago, covering TP4, explicaciones con

costo confundido con tiempo. Bitacoras DUIA por TP.

Anexo - Capturas de cierre de gaps

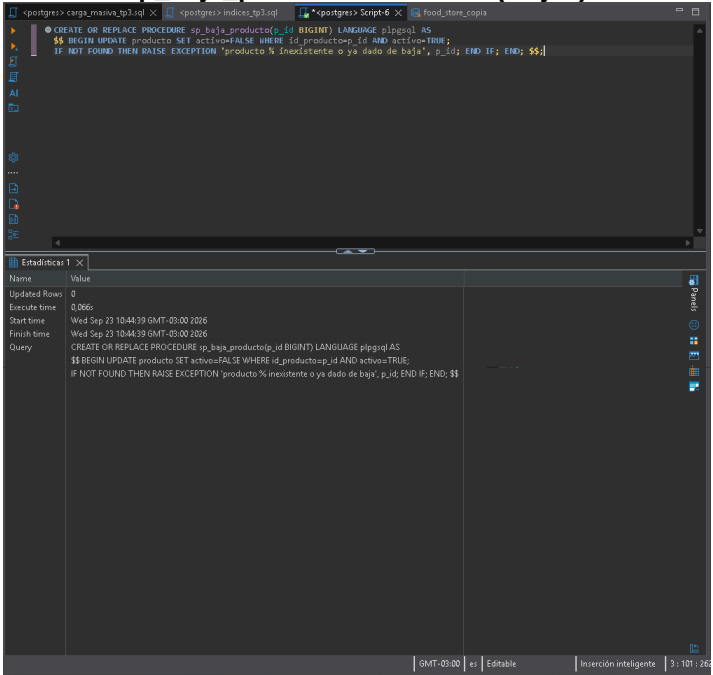
H1 - HAVING facturacion por categoria: 6 filas sobre 20M (Obj. 5).



```
SELECT c.nombre AS categoria, SUM(d.cantidad*d.precio_unitario) AS facturacion
FROM categoria c JOIN producto p ON p.id_categoria=c.id_categoria
JOIN pedido_producto d ON d.id_producto=p.id_producto
GROUP BY c.nombre HAVING SUM(d.cantidad*d.precio_unitario) > 20000000
ORDER BY facturacion DESC;
```

	AZ categoria	facturacion
1	Categoria masiva 4	50.000.000
2	Categoria masiva 9	50.000.000
3	Categoria masiva 3	40.000.000
4	Categoria masiva 8	40.000.000
5	Categoria masiva 2	30.000.000
6	Categoria masiva 7	30.000.000

B0 - CREATE PROCEDURE sp_baja_producto en 0.066 s (Obj. 6).



```
CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS
$$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE;
IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; $$;
```

Etadistica	Value
Name	sp_baja_producto
Updated Rows	0
Excute time	0.066s
Start time	Wed Sep 23 10:44:39 GMT-03:00 2026
Finish time	Wed Sep 23 10:44:39 GMT-03:00 2026
Query	CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS \$\$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE; IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; \$\$

B1 - CALL sp_baja_producto(3): vista pasa a 50001 (Obj. 9).

Script: C:\Users\Fabri\OneDrive\Documents\Default Project\sql\carga_masiva_tp3.sql

Connection: postgres

Type: PostgreSQL

URL: jdbc:postgresql://localhost:5432/postgres

Database: food_store_copia

Schema: public

Script-6

```
BEGIN;  
CALL sp_baja_producto(3);  
SELECT * FROM v_productos_vigentes WHERE id_producto=3;  
SELECT COUNT(*) FROM v_productos_vigentes;  
ROLLBACK;
```

v_productos_vigentes 2

v_productos_vigentes 2 (2)

Estadísticas 2

1

count

50.001

GMT-03:00

Editable

Inserción inteligente

5:10:145

B2 - La vista ya no trae el producto 3: 0 filas (Obj. 9).

Script: C:\Users\Fabri\OneDrive\Documents\Default Project\sql\carga_masiva_tp3.sql

Connection: postgres

Type: PostgreSQL

URL: jdbc:postgresql://localhost:5432/postgres

Database: food_store_copia

Schema: public

Script-6

```
BEGIN;  
CALL sp_baja_producto(3);  
SELECT * FROM v_productos_vigentes WHERE id_producto=3;  
SELECT COUNT(*) FROM v_productos_vigentes;  
ROLLBACK;
```

v_productos_vigentes 2

v_productos_vigentes 2 (2)

Estadísticas 2

id_producto

nombre_producto

precio_lista

stock

nombre_categoria

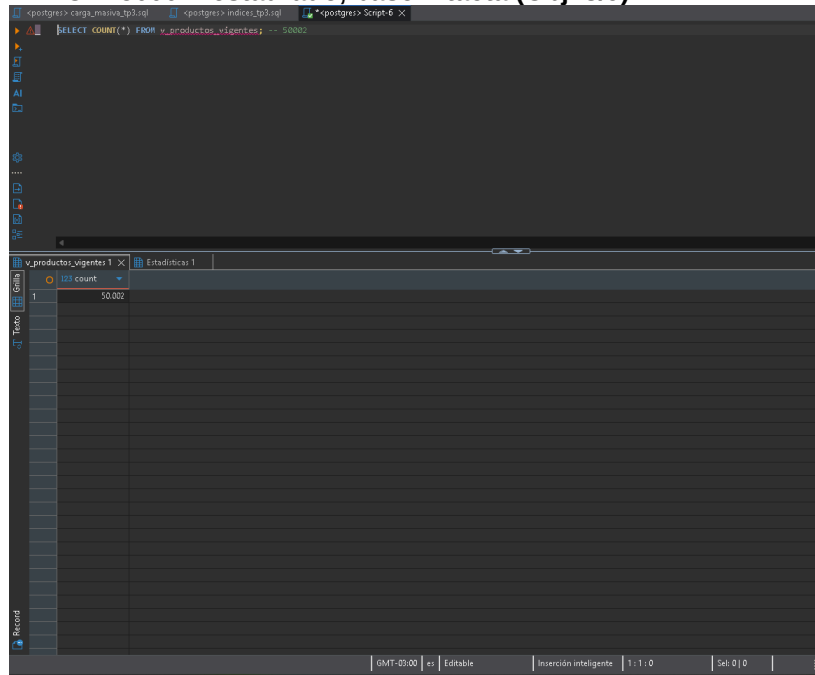
GMT-03:00

Editable

Inserción inteligente

5:10:145

B3 - Tras ROLLBACK: 50002 restaurado, base intacta (Obj. 8/9).



The screenshot shows a PostgreSQL IDE interface. The top window, titled "<postgres> Script-6 X", contains the SQL command `ROLLBACK;`. Below it, the "v_productos_vigentes T" table is selected, and the "count" aggregate function is applied. The results window displays a single row with the value 50002.

count
50002